PROYECTO FINAL · DEEP LEARNING Y SISTEMAS INTELIGENTES 2026 · UVG

¿Mantenemos este modelo en producción?

Monitoreo transaccional del Banco del Altiplano: API de riesgo de fraude con campeón y retador

Andrés Mazariegos · June Herrera · *[Integrante 3]*

Modelo de origen: Proyecto 1 · Prof. Kevin Recinos · noviembre 2026

1 · PROBLEMÁTICA

Cada transacción se decide en milisegundos, sin saber todavía si es fraude

DEJAR PASAR UN FRAUDE

Q4,200

BLOQUEAR A UN CLIENTE LEGÍTIMO

Q180

Un error es 23 veces más caro que el otro: el umbral no puede estar en 50 %.

LA DECISIÓN QUE DEPENDE DEL MODELO

PROBABILIDAD CALIBRADA	ACCIÓN	QUIÉN
≥ 1.6 %	Bloquear	automático
0.23 % – 1.6 %	Revisar	analista (≤ 1 % del tráfico)
< 0.23 %	Aprobar	automático

USUARIOS

Sistema autorizador — llama a la API por transacción, necesita p95 < 100 ms.
Analista de riesgos — atiende la cola de revisión y necesita saber *por qué*.
Este comité — decide con el panel si el modelo se mantiene, se reentrena o se reemplaza.

Hipótesis escrita antes de entrenar; paridad probada antes de desplegar

ETAPA	FECHA	RESPONSABLE
Elegir modelo (P1 sobre P2 y P3)	8 nov	Trío
Congelar hipótesis y criterio	8 nov 20:26	A. Mazariegos
Reentrenar K = 3, paquete de producción	8 nov	J. Herrera
API, demo, registro, monitoreo	8 nov	A. Mazariegos
Pruebas de contrato y paridad	8 nov	*Integrante 3*
Despliegue y tráfico de prueba	9 nov	*Integrante 3* · J. Herrera
Monitoreo y versión final	≤ 15 nov	Trío

DATOS

Sintéticos (generador del P1, misma semilla): 419,821 transacciones, 4,000 tarjetas, 1.18 % fraude. Partición **temporal**. Las etiquetas llegan días después (contracargos).

USUARIO

El autorizador envía 7 días de historia. Los analistas revisan ≤ 1 % del tráfico. Costos promedio fijos.

ENTORNO

CPU, 2 vCPU (Hugging Face Spaces gratuito). Registro efímero, exportable a CSV.

Del Proyecto 1 conservamos todo y cambiamos una sola cosa: K = 20 → 3

LO QUE DIJO EL PROYECTO 1 DE SÍ MISMO

Su propia prueba de recorte de historia mostró que el GRU con `K = 20` estaba en su **peor** configuración (0.636 de AUC-PR contra 0.749 con `K = 3`), y que el resultado dependía mucho de la semilla (± 0.068).

POR QUÉ TENDRÍA SENTIDO

El patrón "sondeo y golpe" vive en las últimas transacciones. Con 20 pasos el GRU tenía que aprender a ignorar 17 pasos de relleno o ruido en 30 épocas.

RETADOR — C_K3 (DEEP LEARNING)

últimas 3 transacciones → 7 numéricas + embeddings (rubro 8, canal 4, comercio 16) → **GRU 64** con máscara → dropout 0.3 → ⊕ 30 agregados → Dense 32 → sigmoide · 27,457 parámetros

LÍNEA BASE — A

LightGBM sobre 30 agregados causales, ciegos al orden

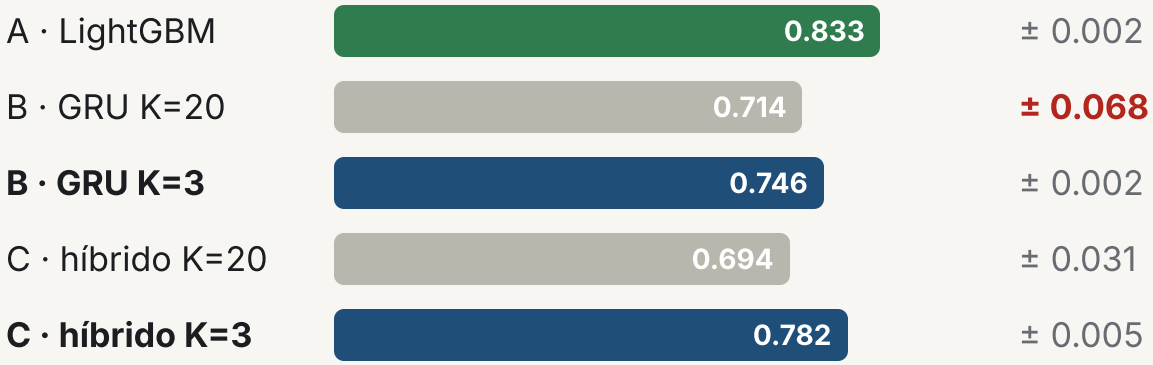
LO QUE NO CAMBIÓ

Datos, partición, features, semillas (7, 13, 29), hiperparámetros. Si algo mejora, es por K.

4 · VALIDACIÓN DE HIPÓTESIS

H1 se cumple; H2 no. Lo importante: el GRU dejó de depender de la suerte

AUC-PR de validación, media de 3 semillas (barra) ± desviación



Hipótesis registradas 02:26 UTC; entrenamiento inició 02:28 UTC y guardó el SHA-256 del archivo de hipótesis.

H1 · GRU K=3 > GRU K=20

Criterio: +0.03 en la media **y** peor semilla > 0.714

+0.032 · peor semilla 0.743 **Se cumple**

H2 · HÍBRIDO K=3 ≥ LIGHTGBM

Criterio: media ≥ A − 0.005

−0.051 **No se cumple**

PERMUTACIÓN (FALSIFICACIÓN)

Barajar la ventana: 0.786 → 0.661. El GRU **sí** usa el orden.

4 · REGLA DE DECISIÓN (CONGELADA) Y PRUEBA, EVALUADA UNA VEZ

LightGBM decide; el GRU híbrido corre en sombra sobre el mismo tráfico

PRUEBA · 13 DÍAS · 703 FRAUDES	AUC-PR	RECALL	COSTO	AHORRO / MES
A — campeón	**0.857**	97.7 %	**Q261 k**	**Q6.02 M**
C_K3 — retador	0.789	94.2 %	Q365 k	Q5.79 M
Sin modelo	—	0 %	Q2.95 M	—

El orden de la tabla es el mismo que en validación: test confirmó la decisión, no la tomó.

POR QUÉ SOMBRA Y NO REEMPLAZO

Ningún secuencial superó a A. Pero el retador ve exactamente lo mismo que el campeón, su decisión se registra y no se ejecuta: el comité puede compararlos en tráfico real sin arriesgar un quetzal.

SI EL COMITÉ DECIDE PROMOVERLO

Se reinicia con `PROMOVER_RETADOR=1`. Sin reentrenar: cada modelo conserva su calibrador y su umbral.

Datos → entrenamiento → servicio → monitoreo

Datos
Generador P1 (semilla fija) · partición temporal train / val / test

→

Entrenamiento
Features del P1 · LightGBM ×3 · GRU ×3 · regla congelada · calibración + u* en val · ~10 min CPU

→

Artefactos
modelos · scalers · vocabularios · umbrales · referencia PSI · metadata con hash de hipótesis

↓

Servicio · Docker en Hugging Face Spaces
FastAPI + Uvicorn (2 procesos) · `POST /predecir` · campeón decide, retador en sombra · motivos SHAP para el analista · validación de entradas (422)

→

Registro
SQLite: cada predicción + etiqueta que llega después (`/retroalimentacion`)

→

Monitoreo
`/monitor`: latencia, PSI de puntaje y entradas en ventana móvil, costo real de campeón vs. retador

Clave del diseño: el servicio **no reimplementa ninguna feature**. Arma un mini-DataFrame con la historia + la transacción y llama a las *mismas* funciones del entrenamiento.

Keras 3 (torch) · LightGBM · FastAPI · Pydantic · SQLite · Chart.js local · Docker `python:3.13-slim` + torch CPU

75 pruebas automáticas; la más importante atrapó un bug antes de desplegar

PARIDAD ENTRENAMIENTO ↔ SERVICIO

40 transacciones reales del periodo de prueba (15 fraudes), puntuadas por el pipeline por lotes y por la API. Diferencia máxima permitida: **0.00001**, en ambos modelos.

Primera corrida: **falló** — timestamps con y sin microsegundos se parseaban con un solo formato. Corregido: **pasa**

CONTRATO DE LA API

- Decisión siempre coherente con los umbrales
- 422 ante canal, rubro, monto o fecha inválidos
- 422 si la transacción es anterior a su historia
- Retroalimentación, exportación CSV, borrado protegido

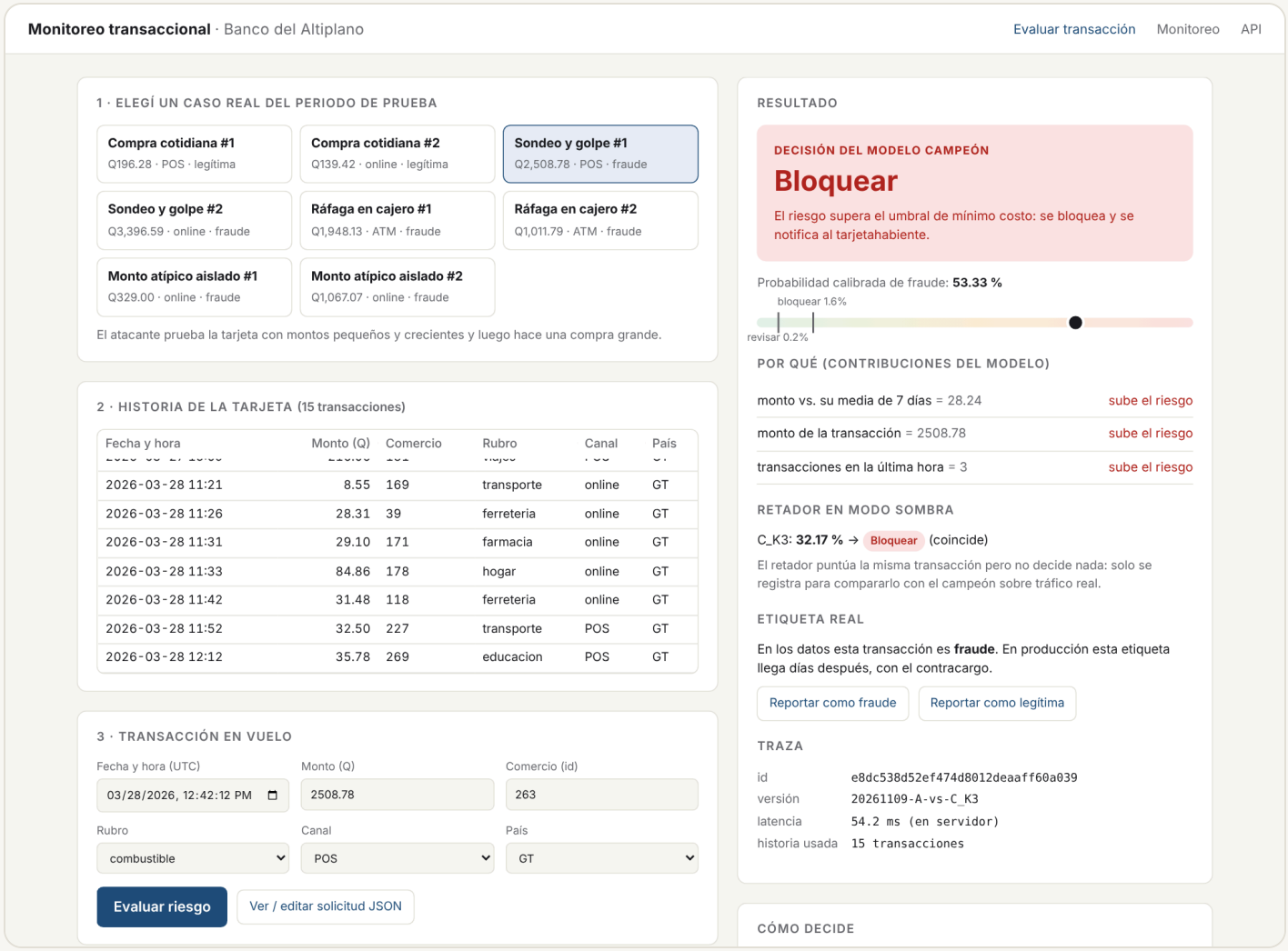
HISTORIA RECORTADA

Mandar toda la historia o solo 7 días da el mismo puntaje → el contrato de 7 días es suficiente.

MONITOREO

El PSI calla con tráfico estable y alerta con montos inflados o un canal nuevo.

Demo: un "sondeo y golpe" real del periodo de prueba



LO QUE VE EL ANALISTA

Varias compras pequeñas en línea en 20 minutos (Q8, Q28, Q29, Q85…) y, una hora después, Q2,509 en una gasolinera. Riesgo 53 % → **Bloquear**.

POR QUÉ

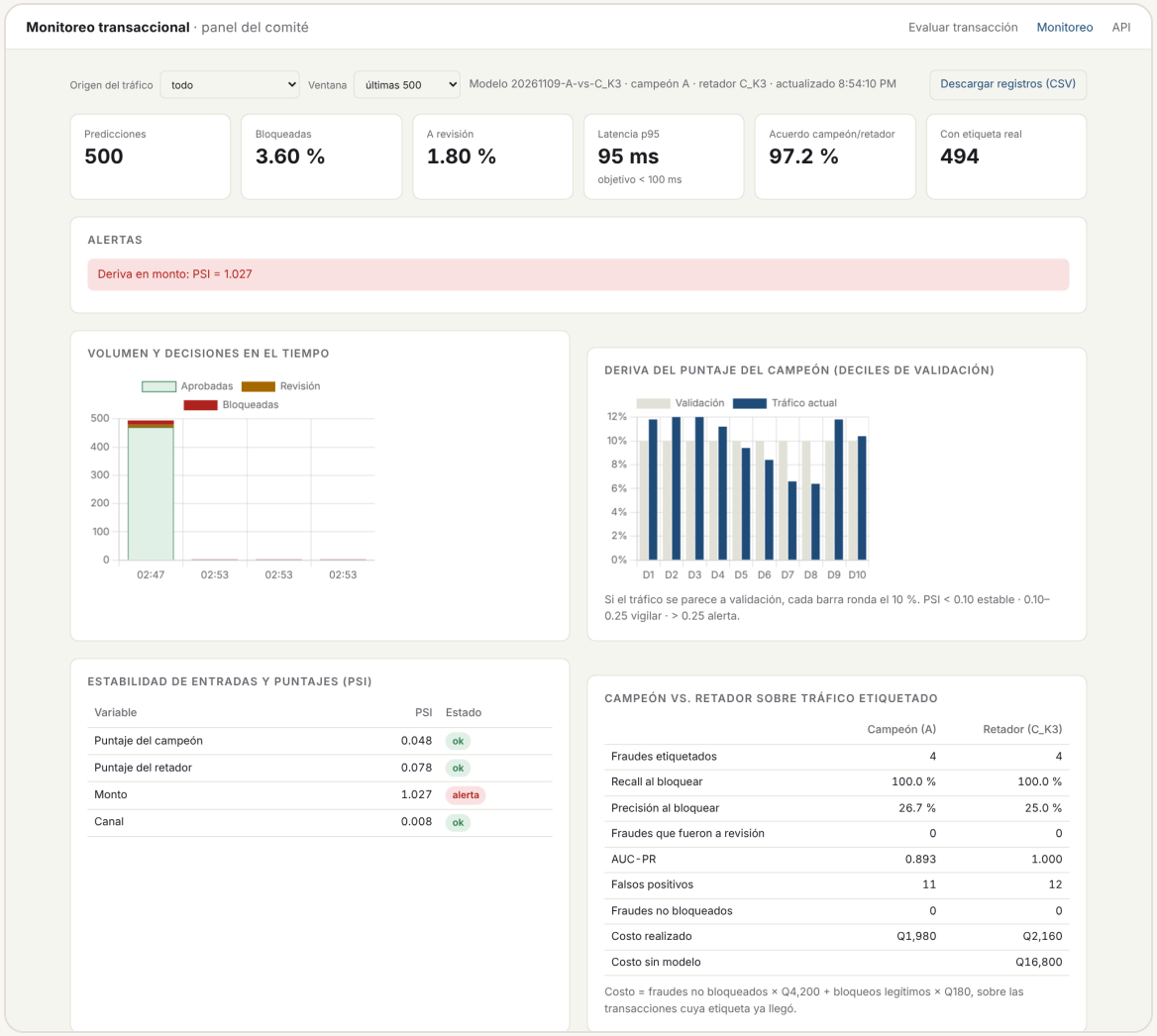
Monto 28× su media de 7 días · 3 transacciones en la última hora.

RETADOR

32 % → también bloquearía.

URL: *<usuario>-monitoreo-transaccional.hf.space* · `/docs` para la API

2,500 transacciones reproducidas: 0 errores, y el monitoreo encontró dos cosas



2,000 TX · 43 FRAUDES	RECALL	COSTO
Campeón A	100 %	**Q6,300**
Retador C_K3	97.7 %	Q9,780
Sin modelo	0 %	Q180,600

1 · LATENCIA

Con 1 proceso y 4 clientes: p95 = 252 ms ❌. Causa: GIL de Python. Con 2 procesos: 45–51 tx/s, p95 95–139 ms.

2 · DERIVA (MONTOS ×2.5)

PSI del monto = **1.02 → alerta**; PSI del puntaje = 0.05. Las entradas cambiaron, el modelo no: usa montos *relativos*. Alerta de entrada ≠ modelo roto.

RECOMENDACIÓN AL COMITÉ

Mantener en producción, con el GRU en sombra cuatro semanas

DECISIÓN QUE PROPONEMOS

- **Mantener** a LightGBM como campeón: ahorra ~Q6 M/mes frente a no tener modelo.
- **Mantener** el GRU híbrido K=3 en sombra y revisarlo cada semana en el panel.
- **Promover** al retador solo si, con ≥ 200 fraudes etiquetados, su costo realizado es menor que el del campeón.

DISPARADORES DE REENTRENAMIENTO

PSI del puntaje > 0.25 · tasa de bloqueo > 3× la esperada · recall al bloquear < 90 %

RIESGOS QUE NO ESCONDEMOS

- **Datos sintéticos**: probamos el sistema, no que el patrón exista así en el banco.
- Calibrador y umbral heredan el optimismo de validación.
- Costos fijos: en la realidad un FN escala con el monto.
- Registro efímero en el plan gratuito; sin autenticación en la API.

SIGUIENTE ITERACIÓN

Ensamble de A y C_K3 (promedio de probabilidades calibradas), validado con el mismo protocolo.